

Software Requirement **Specification**

for

MedSync Clinic Appointment
And Treatment Management
System

<Version 1.0>

2026.07.28

Prepared by - Group 01

- Wijerathne W.D.D - 220715T
- Jayalath K.D - 240264C
- Sulakshana D.G - 240634L
- Sivakumaran S - 240628X
- Kumara D.G.C.M - 240362B
- Fernando R.V. - 240167G

Submitted in partial fulfillment
Of the requirements of
CS3043 Database Systems

Table of Contents

- **1 Introduction**
 - [1.1 Purpose](#)
 - [1.2 Scope](#)
 - [1.3 Definitions, Acronyms and Abbreviations](#)
 - [1.4 References](#)
 - [1.5 Overview of the Document](#)
- **2 Overall Description**
 - [2.1 Product Perspective](#)
 - [2.1.1 System context](#)
 - [2.2 Product Functions](#)
 - [2.3 User Characteristics](#)
 - [2.4 Constraints](#)
 - [2.5 Assumptions and Dependencies](#)
- **3 External Interface Requirements**
 - [3.1 User Interfaces](#)
 - [3.1.1 Operational Interface \(FastAPI Swagger UI\)](#)
 - [3.1.2 Reporting Interface \(Dashboard\)](#)
 - [3.2 Hardware Interfaces](#)
 - [3.3 Software Interfaces](#)
 - [3.4 Communications Interfaces](#)
- **4 System Features**
 - [4.1 Branch Management](#)
 - [4.1.1 Manage Clinic Branches](#)
 - [4.1.2 Assign Branch Manager](#)
 - [4.1.3 View Branch Information](#)
 - [4.2 Staff and Doctor Management](#)
 - [4.2.1 Register Staff Members](#)
 - [4.2.2 Manage Doctor Profiles](#)
 - [4.2.3 Assign Doctors to Branches](#)
 - [4.2.4 Manage Doctor Specializations](#)
 - [4.2.5 View Staff Information](#)
 - [4.3 Patient Registration and Medical Records](#)
 - [4.3.1 Register New Patient](#)
 - [4.3.2 Update Patient Information](#)
 - [4.3.3 Maintain Emergency Contact Details](#)
 - [4.3.4 Maintain Health Insurance Information](#)
 - [4.3.5 Access Patient Records Across Branches](#)
 - [4.4 Appointment Management](#)
 - [4.4.1 Schedule Appointment](#)
 - [4.4.2 Reschedule Appointment](#)
 - [4.4.3 Cancel Appointment](#)
 - [4.4.4 Record Walk-in Appointments](#)

- [4.4.5 Manage Appointment Status](#)
 - [4.4.6 Prevent Appointment Conflicts](#)
- [4.5 Treatment and Consultation Management](#)
 - [4.5.1 Record Consultation Notes](#)
 - [4.5.2 Record Treatments](#)
 - [4.5.3 Maintain Treatment Catalogue](#)
 - [4.5.4 Assign Treatments to Appointments](#)
- [4.6 Billing and Payment Management](#)
 - [4.6.1 Generate Invoice](#)
 - [4.6.2 Calculate Treatment Charges](#)
 - [4.6.3 Record Payments](#)
 - [4.6.4 Manage Partial Payments](#)
 - [4.6.5 Calculate Outstanding Balance](#)
- [4.7 Insurance Claim Management](#)
 - [4.7.1 Verify Patient Insurance](#)
 - [4.7.2 Calculate Insurance Coverage](#)
 - [4.7.3 Process Insurance Claims](#)
 - [4.7.4 Calculate Patient Out-of-Pocket Payment](#)
- [4.8 Reporting and Analytics](#)
 - [4.8.1 Branch-wise Appointment Summary](#)
 - [4.8.2 Doctor Revenue Report](#)
 - [4.8.3 Outstanding Balance Report](#)
 - [4.8.4 Treatment Category Report](#)
 - [4.8.5 Insurance Coverage Report](#)
- [4.9 Database Integrity and Transaction Management](#)
 - [4.9.1 Keys and Constraints](#)
 - [4.9.2 Data Validation Rules](#)
 - [4.9.3 Referential Integrity](#)
 - [4.9.4 Indexing Strategy](#)
 - [4.9.5 Stored Procedures](#)
 - [4.9.6 Database Functions](#)
 - [4.9.7 Database Triggers](#)
 - [4.9.8 Transaction Management and ACID Properties](#)
- [4.10 Security and Access Control](#)
 - [4.10.1 User Authentication](#)
 - [4.10.2 Role-Based Access Control](#)
 - [4.10.3 Audit Logging](#)
 - [4.10.4 Data Privacy](#)
- **[5 Non-Functional Requirements \(NFR\) Specification](#)**
 - [5.1 Introduction to Non-Functional Requirements in DBMS](#)
 - [5.1.1 Functional vs. Non-Functional Requirements Scope](#)
 - [5.1.2 Multi-Branch Context & NFR Rationale](#)
 - [5.2 Comprehensive Non-Functional Requirements \(Categorised\)](#)
 - [5.2.1 Category: Performance & Query Latency](#)
 - [5.2.2 Category: Data Integrity & ACID Guarantees](#)
 - [5.2.3 Category: Data Security, Privacy & Data Minimisation](#)
 - [5.2.4 Category: Reliability & Durability](#)

- [5.2.5 Category: Usability & QA Testability](#)
- [5.2.6 Category: Maintainability & Schema Scalability](#)
- [5.3 Summary NFR Metrics Table](#)

- **[6 Other Requirements and Appendixes](#)**

- [6.1 Traceability Matrix](#)
- [6.2 Sample Report Layouts](#)
 - [6.2.1 Branch-wise Appointment Summary \(FR-RA-01, FR-RA-02\)](#)
 - [6.2.2 Doctor Revenue Report \(FR-RA-03, FR-RA-04\)](#)
 - [6.2.3 Outstanding Balance Report \(FR-RA-05, FR-RA-06\)](#)
 - [6.2.4 Treatment Category Report \(FR-RA-07, FR-RA-08\)](#)
 - [6.2.5 Insurance Coverage Report \(FR-RA-09, FR-RA-10\)](#)
- [6.3 Appendix A - Sample QA Test Cases](#)
- [6.4 Additional Requirements Not Elsewhere Categorized](#)
- [6.5 Supplementary Glossary](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete set of requirements for the Clinic Appointment and Treatment Management System (CATMS), to be developed for MedSync, a medium-scale multi-specialty clinic operating branches in Colombo, Kandy and Galle. MedSync currently administers its appointments, doctor rosters and billing through a combination of paper registers and spreadsheets maintained separately at each branch. As patient volumes and the number of resident specialists have grown, this arrangement has produced double-booked consultation slots, patient records that cannot be retrieved when a patient presents at a branch other than the one at which they first registered, and billing figures that cannot be reconciled at the end of a trading day.

The purpose of this document is therefore threefold. First, it records an unambiguous and verifiable statement of what CATMS must do, so that the development team and the client hold the same understanding of the delivered system. Second, it serves as the contractual baseline against which the Phase 1 deliverable will be accepted or rejected. Third and of particular importance for this project it specifies the database-level obligations of the system: the entities and their relationships, the primary and foreign keys that preserve referential integrity, the indexes that make the required reports tractable, and the stored procedures, functions and triggers through which the atomicity, consistency, isolation and durability guarantees of the database are enforced.

The intended audience comprises the MedSync management team and branch managers, who should read Sections 1 and 2; the development team, who should read the document in full; the independent QA team, who will derive test cases from Section 3 and Appendix A; and the module coordinator assessing the academic deliverable.

1.2 Scope

The software product specified by this document is named CATMS. Phase 1, which this SRS covers in its entirety, delivers a correct, normalised and fully populated relational database together with a thin user interface sufficient for the QA team to exercise the system's key features. The emphasis throughout is on database correctness rather than interface sophistication

In Scope

- Registration and maintenance of branches, medical and non-medical staff, and the assignment of a branch manager to each branch.
- Maintenance of a doctor register in which each doctor is attached to exactly one branch but may practise in more than one specialty.
- Registration of patients at any branch, with a single logical patient record that is retrievable from every branch, including personal details, an emergency contact and health insurance details.
- Booking, rescheduling and cancellation of appointments between a patient and a doctor for a specified time slot, with hard prevention of overlapping appointments for the same doctor.
- Creation of emergency walk-in appointments directly by clinic staff without a prior booking.

- Recording of consultation notes and one or more catalogued treatments against a completed appointment.
- Maintenance of a treatment catalogue in which every treatment carries a service code, a category and a price.
- Automatic generation of an invoice from the treatments and consultation recorded against a completed appointment.
- Acceptance of full or partial payments against an invoice and continuous tracking of the outstanding due.
- Lodging and settlement of insurance claims for eligible treatments where the patient holds a policy, with partial or full reimbursement according to policy terms.
- Production of the five management reports enumerated in Section 3.2.7.

Out of Scope for Phase 1

- Clinical decision support, prescription of pharmaceuticals, and laboratory result management.
- Real-time electronic interchange with external insurance providers; claims are lodged and settled within CATMS and exported as a file for manual submission.
- Patient-facing self-service booking through a public website or mobile application.
- Payroll, human-resource management and inventory control.
- Migration of the historical paper and spreadsheet records; the database is populated with representative dummy data for demonstration purposes.

Benefit Statement

On delivery, MedSync will hold a single authoritative record of every appointment, treatment and payment across all three branches. Double-booking becomes impossible at the database level rather than by staff vigilance, outstanding patient balances become visible on demand instead of being reconstructed at month end, and each of the five management reports is produced from a single query against indexed tables.

1.3 Definitions, Acronyms and Abbreviations

The following terms carry the stated meaning throughout this document. Where a term is used in a requirement statement it is to be interpreted strictly as defined here.

Definitions

Term	Definition
Appointment	A reserved interval of a named doctor's time, at a named branch, allocated to a named patient. An appointment occupies exactly one contiguous time slot and carries exactly one status at any instant.
Walk-in	An appointment created by clinic staff at the moment of the patient's arrival, without a prior booking. A walk-in is a normal appointment distinguished only by the is_walkin flag.

Slot	The unit of doctor availability. MedSync operates fifteen-minute slots; an appointment consumes one or more consecutive slots.
Overlap	Two appointments for the same doctor overlap when the half-open intervals [start_ts, end_ts) of the two appointments intersect. Cancelled appointments are excluded from overlap evaluation.
Treatment	A billable clinical service drawn from the pre-defined treatment catalogue, such as Consultation, X-Ray, ECG or Injection. Each treatment has a unique service code and a current price.
Consultation	The narrative record — notes and diagnosis — made by a doctor against a completed appointment. Distinct from the billable Consultation treatment item.
Invoice	The financial document generated from a completed appointment. It aggregates the priced treatment lines and carries a running paid amount and a derived status.
Outstanding due	For an invoice, total_amt less paid_amt less the approved insurance reimbursement. For a patient, the sum of that quantity across all their unsettled invoices.
Insurance claim	A request lodged against a patient's policy for reimbursement of eligible treatment lines on an invoice. A claim has a claimed amount and, once adjudicated, an approved amount.
Coverage percentage	The proportion of an eligible treatment's price that the policy reimburses, subject to the policy's annual ceiling.
Out-of-pocket	The portion of an invoice borne by the patient after insurance reimbursement has been applied.
Specialty	A field of medical practice, such as General Medicine, ENT or Paediatrics. A doctor may hold more than one.
Branch manager	The member of staff accountable for a branch. Exactly one per branch.

Acronyms

Acronym	Expansion	Acronym	Expansion
DBMS	Database Management System	RDBMS	Relational Database Management System
DDL	Data Definition Language	SRS	Software Requirements Specification
DML	Data Manipulation Language	TPS	Transactions Per Second
ERD	Entity–Relationship Diagram	UI	User Interface

FK	Foreign Key	3NF	Third Normal Form
FR	Functional Requirement	BCNF	Boyce–Codd Normal Form
NFR	Non-Functional Requirement	QA	Quality Assurance

1.4 References

Ref.	Reference
[1]	IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, Institute of Electrical and Electronics Engineers, 1998.
[2]	IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
[3]	MedSync Client Brief, Project 1 — Clinic Appointment and Treatment Management System, issued to the design team
[4]	R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson.
[5]	A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill
[6]	ISO/IEC 9075:2016, Information technology — Database languages — SQL
[7]	Sri Lanka Personal Data Protection Act, No. 9 of 2022 — obligations relating to the processing of health data.
[8]	Sri Lanka Medical Council, Guidelines on Medical Records and Record Keeping.
[9]	ISO/IEC 25010:2011, Systems and software Quality Requirements and Evaluation (SQuARE) — used to structure Section 3.4

1.5 Overview of the Document

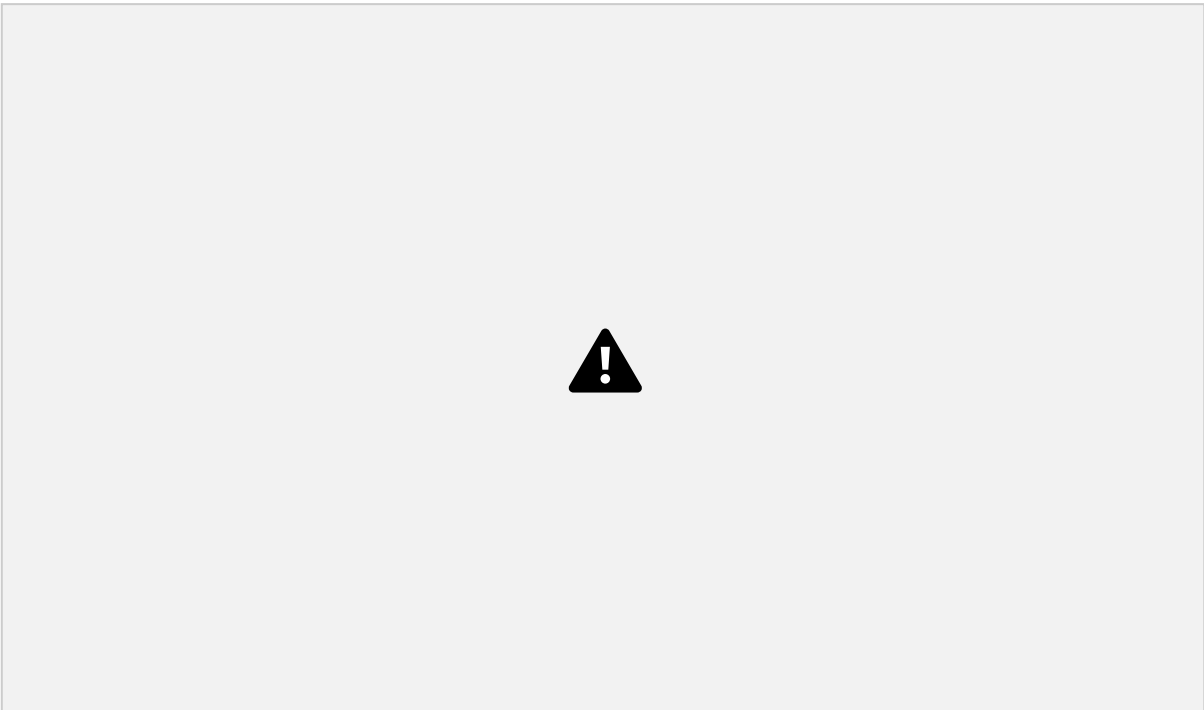
The remainder of this document follows the structure recommended by IEEE 830-1998. Section 2 gives a narrative description of the product: where it sits in relation to the systems and people around it, what it does in broad terms, who uses it, the conceptual data model that underpins everything else, and the constraints and assumptions under which the team is working. Section 2 is written to be intelligible to a reader without a database background. Section 3 contains the requirements proper. Each functional requirement is uniquely identified, assigned a priority, and stated in a form that a tester can pass or fail without recourse to opinion. Section 3.3 deals specifically with the database obligations — keys, indexes, procedures, functions, triggers and the ACID analysis that justifies them. Section 3.4 states the non-functional requirements. Section 4 contains the traceability matrix that maps each clause of the client brief to the requirements that satisfy it, together with sample report layouts.

2. Overall Description

2.1 Product Perspective

CATMS is a self-contained, new build system which replaces the paper appointment registers and the pre-branch billing spreadsheets currently in use. It exchanges information with a number of external actors and systems as show below

2.1.1 System context



As shown above, CATMS acts as the central system through which all clinic operations flow. The **Receptionist/Staff** handle patient bookings and emergency walk-ins, the **Doctor** records clinical notes and treatments, and both interact with the core modules, Appointment Scheduling, Clinical Notes & Treatment, and Billing & Payment. Each module reads from and writes to the shared **MedSync Database**. The Reporting Module draws on this data to generate the management reports (branch summaries, doctor revenue, outstanding balances, and insurance breakdowns) requested by the **Branch Manager**. The **Patient** is treated as a data subject rather than an active user, since all records are created and managed on their behalf by staff.

2.2 Product Functions

At the highest level CATMS performs seven groups of functions. The table below is a summary map, not a specification.

Group	Functions Performed	Principle Entities
1. Branch and Staff	• Register branches • appoint exactly one branch manager per branch • maintain medical and non-medical staff • attach each doctor to a branch and to one or more specialties.	• BRANCH • STAFF • DOCTOR • SPECIALTY • DOCTOR_SPECIALTY
2. Patient Registration	• Register a patient at any branch • Maintain personal details, emergency contact and insurance policy • retrieve the record from any branch • prevent duplicate registration on national identity number.	• PATIENT • EMERGENCY_CONTACT • INSURANCE_POLICY
3. Appointment Scheduling	• Book an appointment for a patient with a doctor in a time slot • reject any overlap on the same doctor • Reschedule • Cancel • create emergency walk-ins	• APPOINTMENT • APPOINTMENT_AUDIT • ROOM
4. Treatment Recording	• Mark an appointment complete • Record consultation notes and diagnosis • prescribe one or more catalogued treatments • maintain the treatment catalogue with service codes, categories and prices.	• CONSULTATION • APPT_TREATMENT • TREATMENT • TREATMENT_CATEGORY
5. Billing	• Generate an invoice from a completed appointment • accept full or partial payments • track the outstanding due • Prevent overpayment.	• INVOICE • PAYMENT
6. Insurance	• Determine treatment eligibility against the patient's policy • lodge a claim • record the adjudicated	• INSURANCE_POLICY • INSURANCE_CLAIM • INVOICE

	- approved amount - apportion the invoice between reimbursement and out-of-pocket.	
7. Reporting	Produce the five management reports required by the client brief.	All

2.3 User Characteristics

Six user classes will interact with CATMS. Their differing technical proficiency and frequency of use shape the interface and permission requirements.

User class	Size & Frequency	Technical Proficiency	Privileges Required
Receptionist	- Approximately four per branch - the heaviest users in the system continuously through the working day.	- Low to moderate. - Comfortable with data entry - no database knowledge. - Turnover is high, so the system must be learnable within a single shift.	- Create and read patients - create, reschedule and cancel appointments - create walk-ins. - No access to clinical notes or to financial totals other than the amount due.
Doctor	- Twelve to eighteen across all branches - Several short sessions per day between consultations.	- Moderate. - Impatient with slow or multi-step interface - will abandon the system if recording a consultation takes longer than writing it.	- Read the day's own appointments and the full history of the patient in front of them - mark appointments complete - write consultation notes and prescribe treatments.
Billing Officer	- Two per branch - continuous use during the afternoon settlement period.	- Moderate to high. - Numerate, - accustomed to spreadsheet - Will detect arithmetic discrepancies immediately.	- Generate invoices - accept payments - lodge and settle insurance claims - read all financial data for their own branch.

Branch Manager	• One per branch • Daily use • principally at the start and end of the day.	• Moderate. • Interested in aggregates rather than individual records.	• Read all reports for their own branch • read staff rosters • no write access to clinical or financial records.
System Administrator	• One • occasional use.	• High. • Fluent in SQL and system administration.	• Full schema access • user and role management • backup and restore. All actions written to the audit log.
QA Tester	• Three • intensive use during the acceptance window only.	• High. • Will deliberately attempt to violate every stated constraint.	• Full functional access against the test dataset • Read access to audit tables to confirm triggers fired.

2.4 Constraints

ID	Constraint	Consequence for Design
C-1	The delivered artefact must be a relational database NoSQL or document stores are not acceptable for this phase.	All modelling is relational
C-2	Business rules protecting integrity must be demonstrably enforced by procedures, functions and triggers, per the client brief.	Rules are implemented in the data tier and mirrored, not replaced, in the application tier.
C-3	The user interface exists only to allow QA to exercise key features.	No effort is budgeted for visual design, no usability requirement is stated beyond learnability.

C-4	Dummy data must be loaded before demonstration, and may be inserted by SQL rather than through the UI.	A seed script is a deliverable, the UI need not support bulk entry.
C-5	Patient health data is regulated under the Personal Data Protection Act No. 9 of 2022.	Encryption at rest, access logging and a defined retention period are mandatory, not optional.
C-6	Currency is Sri Lankan Rupees throughout; all monetary columns are DECIMAL(12,2).	Floating-point types are prohibited for money. Rounding is half-up to two decimals.
C-7	Times are stored in Asia/Colombo local time with no daylight saving adjustment.	Interval arithmetic for overlap detection is straightforward. no timezone conversion is needed.
C-8	The team is four students working part-time over a single semester.	Scope is bounded
C-9	The system must run on either MySQL 8.0 or PostgreSQL 15.	Engine-specific syntax is confined to the procedure layer so that the schema remains portable.

2.5 Assumptions and Dependencies

The client brief explicitly instructs the team to gain domain understanding and to make reasoned assumptions where information is not supplied, for example on treatment prices and insurance terms. Every such assumption is recorded below. Each carries an impact rating indicating what would need to change if the assumption proved false.

ID	Assumption	Basis	Impact if False
A-1	The clinic operates 08:00–20:00 Monday to Saturday and 08:00–13:00 on Sunday.	Typical Sri Lankan private clinic hours.	Low. Opening hours are configuration data, not schema.

A-2	Appointment slots are fifteen minutes, a standard consultation occupies two slots.	Common practice for multi-specialty outpatient clinics.	Low. Slot length is a parameter, overlap logic uses actual timestamps and is unaffected.
A-3	Treatment prices are as follows: • Consultation LKR 2,500 • X-Ray LKR 3,500 • ECG LKR 4,000 • Injection LKR 1,200 • Dressing LKR 900 • Nebulisation LKR 1,500	Indicative private-sector rates; the brief permits assumed prices.	Low. Prices are catalogue rows and are expected to change; historical invoices are protected by storing unit price on the line.
A-4	Insurance policies reimburse a flat coverage percentage between 50% and 100% of eligible treatments, capped by an annual ceiling.	Simplification of typical Sri Lankan health cover.	Medium. A per-treatment or tiered schedule would require an additional POLICY_COVERAGE_RULE entity.
A-5	Consultation fees are always patient-borne and never reimbursed.	Common exclusion in outpatient policies.	Low. Governed by an is_claimable flag on TREATMENT.
A-6	A patient is identified uniquely by national identity number, minors without one are identified by a guardian's number plus date of birth.	Standard Sri Lankan practice.	Medium. Duplicate detection logic would need revision.
A-7	A walk-in is scheduled and completed within the same working day.	Definition of a walk-in.	Low. No schema impact; affects reporting windows only.
A-8	Cancellation is permitted up to two hours before the appointment start, later cancellation is recorded as a no-show.	Reasonable clinic policy.	Low. The threshold is a configuration value read by the status-transition trigger.
A-9	An invoice is raised per appointment, not per patient visit, a patient seeing two doctors in one visit receives two invoices.	Follows from the one-to-one appointment–invoice rule.	Medium. Consolidated billing would require a VISIT entity above APPOINTMENT.
A-10	The leased line between branches has at least 99% availability during working hours.	Standard business-grade service level.	High. Sustained outages would force an offline booking mode with later reconciliation, which is a substantial addition.
A-11	Doctors do not share consulting rooms Simultaneously, room double-booking is	Physical constraint.	Low. A second unique constraint on ROOM covers it.

	prevented alongside doctor double-booking.		
--	--	--	--

3. External Interface Requirements

This section specifies the external interfaces through which CATMS exchanges information with its users and with the hardware, software and network environment in which it operates, per IEEE Std 830-1998. Consistent with constraint C-3, the emphasis of Phase 1 remains database correctness rather than interface sophistication; the interfaces below are accordingly kept thin, and are described only to the depth needed for the QA team to exercise every functional requirement in Section 4.

3.1 User Interfaces

CATMS presents two user-facing surfaces, both generated directly from the backend rather than hand-built, so that neither can silently drift out of sync with what the database actually enforces.

3.1.1 Operational Interface (FastAPI Swagger UI)

A browser-based, auto-generated Swagger/OpenAPI interface (served at /docs) exposes every system feature in Section 4 as an individually callable endpoint. Each endpoint wraps a stored procedure or function, so a Receptionist, Doctor, Billing Officer or QA Tester can register a patient, book an appointment, record a treatment, generate an invoice, accept a payment or lodge an insurance claim without any purpose-built screens. This is the primary interface used by the QA team to derive and execute test cases against Section 3 and Appendix A of this document.

3.1.2 Reporting Interface (Dashboard)

A lightweight Streamlit or React dashboard presents the five management reports required by the client brief (Section 4.8) in tabular and chart form. It is intended for the Branch Manager and Billing Officer, neither of whom is expected to hold SQL knowledge, and it executes the same indexed report queries and views defined for the database layer, so the figures shown can never diverge from what the database computes.

Stimulus (User of System Action)	System Response
User opens the Swagger UI at /docs	System displays every available endpoint grouped by module (Branch, Staff, Patient, Appointment, Treatment, Billing, Insurance, Reporting)

User selects an endpoint and supplies the required parameters	System validates the input against the underlying stored procedure's signature before submission
User executes the request	System returns the stored procedure's result, or a human-readable error message if a business rule is violated
Branch Manager or Billing Officer opens the reporting dashboard	System renders the five management reports using the current data, filterable by branch, doctor and date range

- **FR-EI-01:** The system shall provide a browser-based interface (FastAPI Swagger UI at /docs) through which every functional requirement in Section 4 can be invoked and its result inspected without writing code.
- **FR-EI-02:** The system shall provide a dashboard interface presenting the five management reports of Section 4.8 in tabular and chart form to users with no SQL knowledge.
- **FR-EI-03:** The interface shall expose only the actions permitted to the signed-in user's class (Receptionist, Doctor, Billing Officer, Branch Manager, System Administrator, QA Tester), consistent with the privileges.
- **FR-EI-04:** The interface shall present error messages raised by database triggers and procedures to the user verbatim, without substituting a generic error code, per.
- **FR-EI-05:** All monetary values displayed in the interface shall be formatted as Sri Lankan Rupees (LKR) to two decimal places, consistent with constraint.
- **FR-EI-06:** All date and time values displayed in the interface shall be shown in Asia/Colombo local time without daylight-saving adjustment, consistent with constraint.

3.2 Hardware Interfaces

No specialised hardware is introduced by Phase 1. CATMS is designed to run on the general-purpose hardware already present, or reasonably procurable, at each MedSync branch.

- **Client workstations:** standard desktop or laptop computers at reception, consultation rooms and the billing counter at each of the three branches, each running a modern web browser. No specialised peripherals (card readers, barcode scanners) are required for Phase 1.
- **Branch network hardware:** standard office switches and routers connecting branch workstations to the branch local network and, via the inter-branch leased line (Assumption A-10), to the central database server.
- **Database/application server:** a single on-premises or cloud-hosted machine, sized for the demonstration dataset described in Section 2.5, sufficient to run PostgreSQL 15 or MySQL 8.0 alongside the FastAPI backend. No redundant or specialised server hardware is budgeted for Phase 1.
- **FR-EI-07:** The system shall operate on any standard workstation capable of running a modern web browser; no client-side installation beyond the browser shall be required.
- **FR-EI-08:** The system shall not require any specialised hardware peripherals in Phase 1.

3.3 Software Interfaces

The table below identifies the software components CATMS interfaces with and the role each plays.

Component	Role / Interface
PostgreSQL 15 (primary) / MySQL 8.0 (portable alternative)	Relational database engine holding all schema, data, procedures, functions, triggers and views, per constraint C-9. Engine-specific syntax is confined to the procedure layer.
FastAPI (Python) backend	REST API layer that wraps the database's stored procedures and functions; auto-generates the Swagger/OpenAPI interface described in 3.1.1.
Streamlit / React frontend	Reporting dashboard described in 3.1.2, consuming FastAPI endpoints that in turn query dedicated report views.
Operating system (server)	Any OS able to host PostgreSQL/MySQL and Python 3.x (Linux, Windows or macOS); client workstations require only a modern browser and are OS-independent.
pg_dump / mysqldump	Logical backup and restore utility used to satisfy NFR-4.3.

- **FR-EI-09:** The backend shall interface with the database exclusively through defined stored procedures and functions (Sections 4.9.5, 4.9.6), rather than ad-hoc SQL, to preserve a single authoritative source of business logic, per NFR-6.4.
- **FR-EI-10:** The system shall remain portable between PostgreSQL 15 and MySQL 8.0, with any engine-specific change confined to the procedure and trigger layer, per constraint C-9.
- **FR-EI-11:** The reporting dashboard shall retrieve report data exclusively through the dedicated report views described in Section 4.8, never through direct base-table queries, per NFR-3.1.

3.4 Communications Interfaces

- **Inter-branch link:** Colombo, Kandy and Galle are connected to the central database over a leased line (or equivalent private network/VPN connection), assumed to provide at least 99% availability during clinic operating hours (Assumption A-10).
- **Client-server communication:** standard HTTPS over TCP/IP between browser clients and the FastAPI backend, and standard TCP/IP between the FastAPI backend and the database server.
- **External insurance interchange:** no real-time electronic interface with external insurance providers is implemented in Phase 1; claims are lodged and settled within CATMS and exported as a file for manual submission, consistent with the scope statement in Section 1.2.

- **Notifications:** no SMS or email notification interface is required in Phase 1; all communication is confined to the internal MedSync network.
- **FR-EI-12:** All communication between client browsers and the backend shall use HTTPS.
- **FR-EI-13:** The system shall not require a real-time electronic interface with external insurance providers in Phase 1; insurance claims shall be exportable as a file for manual submission, per Section 1.2.
- **FR-EI-14:** The system shall fail safely (informative error, no data corruption) during interruptions to the inter-branch leased line; an offline booking mode remains explicitly out of scope, per Assumption A-10.

4. System Features

This section describes the major functional capabilities of the Clinic Appointment and Treatment Management System (CATMS). Each system feature is presented with its purpose, priority level, functional description, and detailed requirements. The features cover key operations including branch and staff management, patient registration, appointment scheduling, treatment recording, billing, insurance claim processing, reporting, and database integrity management.

4.1 Branch Management

4.1.1 Manage Clinic Branches

1.Description

This feature allows administrators to manage information related to clinic branches. The system maintains details of all branches, including branch identification, location, contact details, and operational information.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects the branch management module	System displays available branch management options
Administrator enters new branch details	System validates the entered information
Administrator submits branch information	System creates a new branch record

Administrator updates existing branch details	System modifies the branch information in the database
Administrator requests branch details	System retrieves and displays branch information

4.Functional Requirements

- **FR-BM-01:** The system shall allow authorized users to add new clinic branches.
- **FR-BM-02:** The system shall allow authorized users to update existing branch information.
- **FR-BM-03:** The system shall maintain unique identification for each branch.
- **FR-BM-04:** The system shall store branch details including branch name, address, and contact information .

4.1.2 Assign Branch Manager

1.Description

This feature allows administrators to assign and manage branch managers responsible for individual clinic branches.

2.Priority: High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects a branch	System displays available managers
Administrator selects a manager	System checks manager availability
Administrator confirms assignment	System assigns the manager to the selected branch
Administrator changes branch manager	System updates the manager assignment

4.Functional Requirements

- **FR-BM-05:** The system shall allow a branch manager to be assigned to each branch.
- **FR-BM-06:** The system shall maintain branch manager details.
- **FR-BM-07:** The system shall prevent assigning multiple managers to the same branch at the same time.

4.1.3 View Branch Information

1.Description

This feature allows authorized users to view details of clinic branches and their associated staff members.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User searches for a branch	System retrieves matching branch records
User selects a branch	System displays branch details
User requests staff information	System displays staff assigned to that branch

4.Functional Requirements

- **FR-BM-08:** The system shall display branch details when requested.
- **FR-BM-09:** The system shall allow users to search branches.
- **FR-BM-10:** The system shall display staff members assigned to each branch.

4.2 Staff and Doctor Management

4.2.1 Register Staff Members

1.Description

This feature allows administrators to register medical and non-medical staff members working at clinic branches.

2.Priority: High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects staff registration	System displays staff registration form
Administrator enters staff details	System validates the information
Administrator submits the form	System creates a new staff record
Administrator views staff list	System displays registered staff members

4.Functional Requirements

- **FR-SDM-01:** The system shall allow registration of staff members.
- **FR-SDM-02:** The system shall store staff details including name, contact information, role, and branch assignment.
- **FR-SDM-03:** The system shall maintain separate records for medical and non-medical staff.

4.2.2 Manage Doctor Profiles

1.Description

This feature manages doctor information including personal details, professional details, and availability information.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects doctor management	System displays doctor records
Administrator enters doctor information	System validates doctor details
Administrator saves doctor profile	System stores doctor information
Administrator updates doctor details	System updates the existing profile

4.Functional Requirements

- **FR-SDM-04:** The system shall maintain doctor profiles.
- **FR-SDM-05:** The system shall store doctor registration details and contact information.
- **FR-SDM-06:** The system shall allow authorized users to update doctor information.

4.2.3 Assign Doctors to Branches

1.Description

This feature manages doctor assignments to clinic branches.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects a doctor	System displays available branches
Administrator selects a branch	System validates assignment rules
Administrator confirms assignment	System creates doctor-branch relationship

4.Functional Requirements

- **FR-SDM-07:** The system shall assign doctors to their respective branches.
- **FR-SDM-08:** The system shall maintain doctor-branch relationships.
- **FR-SDM-09:** The system shall allow doctors to work only under valid branch assignments.

4.2.4 Manage Doctor Specializations

1.Description

This feature allows doctors to be associated with one or more medical specialties .

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator selects specialization management	System displays available specialties
Administrator assigns specialties to a doctor	System validates specialization details
Administrator saves changes	System updates doctor specialization records

4.Functional Requirements

- **FR-SDM-10:** The system shall maintain a list of medical specialties.
- **FR-SDM-11:** The system shall allow a doctor to have multiple specialties.
- **FR-SDM-12:** The system shall allow searching doctors by specialization.

4.2.5 View Staff Information

1.Description

This feature allows authorized users to view staff and doctor information.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User searches staff records	System retrieves matching staff information
User selects a staff member	System displays detailed information

4.Functional Requirements

- **FR-SDM-13:** The system shall allow authorized users to view staff details.
- **FR-SDM-14:** The system shall allow searching staff members by role or branch.

4.3 Patient Registration and Medical Records

4.3.1 Register New Patient

1.Description

This feature allows authorized clinic staff to register new patients in the system. The system creates a centralized patient record containing personal details, contact information, emergency contact details, and other required information.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Receptionist selects patient registration	System displays the patient registration form

module	
Receptionist enters patient personal information	System validates the entered data
Receptionist submits the registration form	System creates a new patient record
Registration is completed successfully	System generates a unique Patient ID and displays confirmation

4.Functional Requirements

- **FR-PRM-01:** The system shall allow authorized staff to register new patients.
- **FR-PRM-02:** The system shall generate a unique Patient ID for each registered patient.
- **FR-PRM-03:** The system shall store patient personal information including name, date of birth, gender, address, and contact details.
- **FR-PRM-04:** The system shall validate mandatory patient information before saving records.
- **FR-PRM-05:** The system shall prevent duplicate patient registrations using unique identification information.

4.3.2 Update Patient Information

1.Description

This feature allows authorized users to update patient details when changes occur, such as changes in contact information or personal details.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff searches for an existing patient	System retrieves patient details
Staff selects update option	System displays editable patient information
Staff modifies information and submits changes	System validates and updates the patient record
Update is completed	System displays successful update confirmation

4.Functional Requirements

- **FR-PRM-06:** The system shall allow authorized users to update patient information.
- **FR-PRM-07:** The system shall maintain the latest patient information.
- **FR-PRM-08:** The system shall record modifications made to patient records.

4.3.3 Maintain Emergency Contact Details

1.Description

This feature manages emergency contact information associated with each patient to support urgent medical situations.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff enters emergency contact details	System validates the information
Staff saves emergency details	System associates emergency contact information with the patient record
Staff requests emergency information	System displays saved contact details

4.Functional Requirements

- **FR-PRM-09:** The system shall allow storage of emergency contact information.
- **FR-PRM-10:** The system shall maintain emergency contact details for each patient.
- **FR-PRM-11:** The system shall allow authorized users to update emergency contact information.

4.3.4 Maintain Health Insurance Information

1.Description

This feature allows the system to store and manage patient insurance information required for insurance claim processing.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
----------------------------------	-----------------

Staff enters insurance details	System validates insurance information
Staff saves insurance information	System links insurance details with the patient record
Staff requests insurance information	System displays available insurance details

4.Functional Requirements

- **FR-PRM-12:** The system shall store patient health insurance information.
- **FR-PRM-13:** The system shall maintain insurance policy details.
- **FR-PRM-14:** The system shall allow authorized users to update insurance information.

4.3.5 Access Patient Records Across Branches

1.Description

This feature allows authorized staff from different branches to access centralized patient records.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff searches for a patient from any branch	System searches the centralized database
Patient record is found	System displays patient information
Staff requests medical history	System retrieves available records

4.Functional Requirements

- **FR-PRM-15:** The system shall provide centralized access to patient records.
- **FR-PRM-16:** The system shall allow authorized staff from all branches to access patient information.
- **FR-PRM-17:** The system shall protect patient records from unauthorized access.

4.4 Appointment Management

4.4.1 Schedule Appointment

1.Description

This feature allows staff to schedule an appointment between a patient and a doctor.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff enters appointment details	System checks doctor availability and creates the appointment.

4.Functional Requirements

- **FR-AM-01:** The system shall allow staff to schedule appointments.
- **FR-AM-02:** The system shall check doctor availability before confirming an appointment.
- **FR-AM-03:** The system shall save appointment details.

4.4.2 Reschedule Appointment

1.Description

This feature allows staff to change the date or time of an existing appointment.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff selects an existing appointment and enters a new date or time	System checks availability and updates the appointment.

4.Functional Requirements

- **FR-AM-04:** The system shall allow appointments to be rescheduled.
- **FR-AM-05:** The system shall verify doctor availability before rescheduling.

4.4.3 Cancel Appointment

1.Description

This feature allows staff to cancel an appointment when necessary.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff selects an appointment and chooses Cancel	System updates the appointment status to Cancelled .

4.Functional Requirements

- **FR-AM-06:** The system shall allow staff to cancel appointments.
- **FR-AM-07:** The system shall update the appointment status to **Cancelled**.

4.4.4 Record Walk-in Appointments

1.Description

This feature allows staff to register patients who visit the clinic without a prior appointment.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff registers a walk-in patient	System creates a walk-in appointment.

4.Functional Requirements

- **FR-AM-08:** The system shall support walk-in appointments.
- **FR-AM-09:** The system shall identify walk-in appointments separately from scheduled appointments.

4.4.5 Manage Appointment Status

1.Description

This feature allows staff to update the status of an appointment.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff updates the appointment status	System saves the new appointment status.

4.Functional Requirements

- **FR-AM-10:** The system shall support the statuses **Scheduled**, **Completed**, and **Cancelled**.
- **FR-AM-11:** The system shall allow appointment status updates.

4.4.6 Prevent Appointment Conflicts

1.Description

This feature prevents two appointments from being booked for the same doctor at the same time.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff selects a doctor and time slot	System checks for existing appointments before confirming the booking.

4.Functional Requirements

- **FR-AM-12:** The system shall prevent overlapping appointments for the same doctor.
- **FR-AM-13:** The system shall display an error message if a time conflict is detected.

4.5 Treatment and Consultation Management

4.5.1 Record Consultation Notes

1.Description

This feature allows doctors to record consultation details after completing patient appointments. Consultation notes may include diagnosis information, observations, and recommendations.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Doctor selects a completed appointment	System displays appointment details
Doctor enters consultation notes	System validates the information
Doctor saves consultation details	System stores consultation records linked to the appointment

4.Functional Requirements

- **FR-TCM-01:** The system shall allow doctors to record consultation notes.
- **FR-TCM-02:** The system shall associate consultation notes with the relevant appointment.
- **FR-TCM-03:** The system shall maintain consultation history .

4.5.2 Record Treatments

1.Description

This feature allows doctors to record treatments provided to patients during completed appointments.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Doctor selects treatment option	System displays available treatments
Doctor selects required treatments	System records selected treatments
Doctor confirms treatment details	System stores treatment information

4.Functional Requirements

- **FR-TCM-04:** The system shall allow doctors to record treatments provided to patients.
- **FR-TCM-05:** The system shall allow multiple treatments to be assigned to one appointment.

- **FR-TCM-06:** The system shall maintain treatment history.

4.5.3 Maintain Treatment Catalogue

1.Description

This feature manages the predefined list of treatments offered by the clinic, including service codes and prices.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Administrator opens treatment catalogue	System displays available treatments
Administrator adds or updates treatment information	System validates treatment details
Administrator saves changes	System updates treatment catalogue

4.Functional Requirements

- **FR-TCM-07:** The system shall maintain a predefined treatment catalogue.
- **FR-TCM-08:** The system shall store treatment names, service codes, and prices.
- **FR-TCM-09:** The system shall allow authorized users to update treatment information.

4.5.4 Assign Treatments to Appointments

1.Description

This feature links completed treatments with specific appointments for medical history and billing purposes.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Doctor selects an appointment	System displays available treatment options

Doctor assigns treatments	System creates treatment-appointment relationships
Treatment assignment is confirmed	System updates appointment records

4.Functional Requirements

- **FR-TCM-10:** The system shall associate treatments with appointments.
- **FR-TCM-11:** The system shall use completed appointments for treatment recording.
- **FR-TCM-12:** The system shall make treatment information available for billing purposes.

4.6 Billing and Payment Management

4.6.1 Generate Invoice

1.Description

This feature allows the system to automatically generate invoices for completed appointments based on consultation charges and provided treatments. The invoice contains the details of services provided, treatment charges, insurance coverage (if applicable), and the final payable amount.

2.Priority : High

3.stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Doctor completes an appointment and records treatments	System retrieves completed appointment details
Billing staff selects invoice generation option	System calculates total charges
System identifies assigned treatments and consultation charges	System generates an invoice
Invoice generation is completed	System stores invoice details and displays invoice information

4.Functional Requirements

- **FR-BPM-01:** The system shall generate invoices for completed appointments.
- **FR-BPM-02:** The system shall include consultation charges and treatment charges in invoices.

- **FR-BPM-03:** The system shall assign a unique invoice ID to each invoice.
- **FR-BPM-04:** The system shall maintain invoice records.
- **FR-BPM-05:** The system shall prevent invoice generation for incomplete or cancelled appointments.

4.6.2 Calculate Treatment Charges

1.Description

This feature calculates charges based on treatments provided to patients. The system retrieves treatment prices from the treatment catalogue and calculates the total service cost.

2.Priority : High

3.stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Completed appointment contains assigned treatments	System retrieves treatment prices
System processes treatment information	System calculates total treatment charges
Calculation is completed	System updates invoice details

4.Functional Requirements

- **FR-BPM-06:** The system shall calculate charges based on assigned treatments.
- **FR-BPM-07:** The system shall retrieve treatment prices from the treatment catalogue.
- **FR-BPM-08:** The system shall calculate the total cost of multiple treatments.
- **FR-BPM-09:** The system shall update invoice amounts when treatment details are modified.

4.6.3 Record Payments

1.Description

This feature allows clinic staff to record payments made by patients and maintain payment history.

2.Priority : High

3.stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Patient makes a payment	Staff enters payment details
Staff submits payment information	System validates payment data
Payment is confirmed	System records the payment and updates invoice status

4.Functional Requirements

- **FR-BPM-10:** The system shall record patient payments.
- **FR-BPM-11:** The system shall store payment date, amount, and payment method.
- **FR-BPM-12:** The system shall maintain payment history for each invoice.
- **FR-BPM-13:** The system shall link payments with corresponding invoices.

4.6.4 Manage Partial Payments

1.Description

This feature allows patients to make partial payments while maintaining the remaining outstanding balance.

2.Priority : High

3.stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Patient makes a partial payment	System records the received amount
Payment is saved	System calculates the remaining balance
Staff views invoice status	System displays paid amount and remaining amount

4.Functional Requirements

- **FR-BPM-14:** The system shall support partial payments.
- **FR-BPM-15:** The system shall allow multiple payments for a single invoice.
- **FR-BPM-16:** The system shall maintain payment history until the invoice is fully settled.

4.6.5 Calculate Outstanding Balance

1.Description

This feature calculates unpaid amounts for patient invoices after considering payments and insurance coverage.

2.Priority : High

3.stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Payment is recorded	System retrieves invoice amount
System compares total amount with received payments	System calculates outstanding balance
User requests balance information	System displays remaining payment amount

4.Functional Requirements

- **FR-BPM-17:** The system shall calculate outstanding balances.
- **FR-BPM-18:** The system shall update outstanding balances after each payment.
- **FR-BPM-19:** The system shall provide outstanding balance information for reporting

4.7 Insurance Claim Management

4.7.1 Verify Patient Insurance

1.Description

This feature verifies patient insurance information before processing insurance claims. The system checks whether the patient has valid insurance coverage.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff selects insurance verification option	System retrieves patient insurance details

System checks insurance validity	System displays insurance status
Insurance information is valid	System allows claim processing

4.Functional Requirements

- **FR-ICM-01:** The system shall verify patient insurance information.
- **FR-ICM-02:** The system shall maintain insurance policy details.
- **FR-ICM-03:** The system shall identify whether a patient is eligible for insurance coverage.

4.7.2 Calculate Insurance Coverage

1.Description

This feature calculates the amount covered by insurance based on policy terms and eligible treatments.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Completed treatment details are submitted	System identifies insurance-eligible treatments
System retrieves insurance policy terms	System calculates reimbursable amount
Calculation is completed	System updates billing information

4.Functional Requirements

- **FR-ICM-04:** The system shall identify treatments covered by insurance.
- **FR-ICM-05:** The system shall calculate insurance reimbursement amounts.
- **FR-ICM-06:** The system shall apply insurance coverage rules based on policy information.

4.7.3 Process Insurance Claims

1.Description

This feature manages insurance claim records and tracks the progress of submitted claims.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Staff submits an insurance claim	System validates claim information
Claim information is verified	System creates a claim record
Claim status changes	System updates claim information

4.Functional Requirements

- **FR-ICM-07:** The system shall allow staff to create insurance claims.
- **FR-ICM-08:** The system shall maintain insurance claim records.
- **FR-ICM-09:** The system shall track claim status.

4.7.4 Calculate Patient Out-of-Pocket Payment

1.Description

This feature calculates the amount that must be paid directly by the patient after applying insurance coverage .

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Insurance coverage calculation is completed	System retrieves covered amount
System compares total charges with insurance amount	System calculates patient payable amount
User views billing information	System displays patient responsibility amount

4.Functional Requirements

- **FR-ICM-10:** The system shall calculate patient out-of-pocket payments.
- **FR-ICM-11:** The system shall update patient payable amounts after insurance processing.
- **FR-ICM-12:** The system shall include insurance adjustments in invoices.

4.8 Reporting and Analytics

4.8.1 Branch-wise Appointment Summary

1.Description

This feature generates a daily summary of appointments for each clinic branch.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Manager selects a branch and date	System displays the appointment summary for the selected branch.

4.Functional Requirements

- **FR-RA-01:** The system shall generate a daily appointment summary for each branch.
- **FR-RA-02:** The system shall display the number of Scheduled, Completed, and Cancelled appointments.

4.8.2 Doctor Revenue Report

1.Description

This feature generates a report showing the revenue earned by each doctor.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Manager selects a doctor and date range	System displays the doctor's revenue report.

4.Functional Requirements

- **FR-RA-03:** The system shall generate doctor-wise revenue reports.
- **FR-RA-04:** The system shall calculate revenue based on completed consultations and treatments.

4.8.3 Outstanding Balance Report

1.Description

This feature displays patients who have unpaid or partially paid bills.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Manager requests the outstanding balance report	System displays patients with outstanding balances.

4.Functional Requirements

- **FR-RA-05:** The system shall generate a report of patients with outstanding balances.
- **FR-RA-06:** The system shall display the remaining amount due for each patient.

4.8.4 Treatment Category Report

1.Description

This feature generates a report showing the number of treatments provided by category over a selected period.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Manager selects a date range	System displays the treatment category report.

4.Functional Requirements

- **FR-RA-07:** The system shall generate treatment reports by category.
- **FR-RA-08:** The system shall allow users to filter reports by date range.

4.8.5 Insurance Coverage Report

1.Description

This feature generates a report comparing insurance-covered amounts with patient out-of-pocket payments.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
Manager requests the insurance coverage report	System displays insurance payments and patient payments.

4.Functional Requirements

- **FR-RA-09:** The system shall generate insurance coverage reports.
- **FR-RA-10:** The system shall display insurance-covered amounts and patient out-of-pocket payments.

4.9 Database Integrity and Transaction Management

4.9.1 Keys and Constraints

1.Description

This feature defines database keys and constraints to maintain relationships between entities and prevent invalid data entry. Primary keys uniquely identify records, while foreign keys maintain relationships between related entities.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
----------------------------------	-----------------

User inserts a new record into the database	System checks primary key and constraint rules
User enters duplicate identification data	System rejects the transaction and displays an error
User creates relationships between entities	System validates foreign key references
User attempts invalid data insertion	System prevents saving incorrect data

4.Functional Requirements

- **FR-DB-01:** The system shall use primary keys to uniquely identify records in each entity.
- **FR-DB-02:** The system shall use foreign keys to maintain relationships between related entities.
- **FR-DB-03:** The system shall enforce unique constraints where duplicate values are not allowed.
- **FR-DB-04:** The system shall enforce NOT NULL constraints for mandatory attributes.
- **FR-DB-05:** The system shall prevent deletion of records that are referenced by other entities.

4.9.2 Data Validation Rules

1.Description

This feature ensures that only valid and accurate information is stored in the database.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User enters data into the system	System validates the entered values
Invalid data is detected	System displays an error message
Valid data is provided	System allows the record to be saved

4.Functional Requirements

- **FR-DB-06:** The system shall validate user-entered data before storing it.
- **FR-DB-07:** The system shall validate required fields.

- **FR-DB-08:** The system shall maintain valid data formats for dates, contact numbers, and identification values.
- **FR-DB-09:** The system shall prevent invalid appointment dates and payment amounts.

4.9.3 Referential Integrity

1.Description

This feature maintains consistency between related database tables by ensuring that relationships between entities remain valid.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User creates a relationship between two entities	System checks referenced records
Referenced record does not exist	System rejects the operation
Related record is updated or deleted	System applies integrity rules

4.Functional Requirements

- **FR-DB-10:** The system shall maintain referential integrity between related tables.
- **FR-DB-11:** The system shall prevent orphan records in the database.
- **FR-DB-12:** The system shall ensure valid relationships between patients, appointments, treatments, invoices, and payments.

4.9.4 Indexing Strategy

1.Description

This feature improves database performance by creating indexes on frequently searched and accessed attributes.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User searches patient information	System uses patient-related indexes to retrieve data efficiently
User searches appointments by date or doctor	System uses appropriate indexes
Database query is executed	System improves query execution performance

4.Functional Requirements

- **FR-DB-13:** The system shall create indexes on frequently searched attributes.
- **FR-DB-14:** The system shall optimize database query performance using indexing.
- **FR-DB-15:** The system shall maintain indexes for important entities such as Patient, Doctor, Appointment, and Invoice.

4.9.5 Stored Procedures

1.Description

This feature uses stored procedures to perform frequently used database operations securely and efficiently.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User performs a database operation	System calls the relevant stored procedure
Stored procedure receives input values	System executes predefined database logic
Operation completes	System returns the result

4.Functional Requirements

- **FR-DB-16:** The system shall use stored procedures for complex database operations.
- **FR-DB-17:** The system shall use stored procedures for appointment booking.
- **FR-DB-18:** The system shall use stored procedures for invoice generation.
- **FR-DB-19:** The system shall use stored procedures to improve database security and consistency.

4.9.6 Database Functions

1.Description

This feature provides reusable database functions for performing calculations and data processing.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
System requires a calculation	System calls the relevant database function
Function receives required values	System performs the calculation
Calculation completes	System returns the calculated result

4.Functional Requirements

- **FR-DB-20:** The system shall provide database functions for calculations.
- **FR-DB-21:** The system shall use functions to calculate outstanding balances.
- **FR-DB-22:** The system shall use functions to calculate insurance coverage amounts.
- **FR-DB-23:** The system shall use functions to calculate treatment charges.

4.9.7 Database Triggers

1.Description

This feature automatically performs actions when specific database events occur. Triggers help maintain consistency and automate repetitive operations.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
A record is inserted, updated, or deleted	Database trigger is activated
Trigger condition is satisfied	System performs the predefined action

Action completes	Database remains consistent
------------------	-----------------------------

4.Functional Requirements

- **FR-DB-24:** The system shall use triggers to maintain database consistency.
- **FR-DB-25:** The system shall automatically update related records when required.
- **FR-DB-26:** The system shall use triggers for maintaining audit information.
- **FR-DB-27:** The system shall prevent invalid database operations using triggers.

4.9.8 Transaction Management and ACID Properties

1.Description

This feature ensures reliable database transactions by maintaining the four ACID properties: Atomicity, Consistency, Isolation, and Durability.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User performs a transaction such as payment or appointment booking	System starts a database transaction
Transaction operations are executed	System maintains consistency rules
Transaction completes successfully	System commits changes permanently
Transaction fails	System rolls back changes

4.Functional Requirements

- **FR-DB-28:** The system shall ensure that a transaction is completed fully or not executed at all.
- **FR-DB-29:** The system shall maintain valid database states before and after transactions.
- **FR-DB-30:** The system shall ensure that concurrent transactions do not interfere with each other.
- **FR-DB-31:** The system shall permanently store committed transactions.

4.10 Security and Access Control

4.10.1 User Authentication

1.Description

This feature verifies the identity of users before allowing access to the system.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User enters login credentials	System validates username and password
Credentials are correct	System grants system access
Credentials are incorrect	System denies access and displays an error

4.Functional Requirements

- **FR-SAC-01:** The system shall authenticate users before providing access.
- **FR-SAC-02:** The system shall store user credentials securely.
- **FR-SAC-03:** The system shall prevent unauthorized login attempts.

4.10.2 Role-Based Access Control

1.Description

This feature controls system access according to user roles such as administrator, doctor, receptionist, and billing staff.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User logs into the system	System identifies the user's role
User accesses a module	System checks permission level
Permission is valid	System allows access
Permission is invalid	System blocks access

4.Functional Requirements

4.10.3 Audit Logging

1.Description

This feature records important user activities for monitoring and accountability purposes.

2.Priority : Medium

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User performs an important action	System records the activity
Data modification occurs	System stores user and timestamp details
Administrator views logs	System displays recorded activities

4.Functional Requirements

4.10.4 Data Privacy

1.Description

This feature ensures that patient medical and personal information is protected from unauthorized access.

2.Priority : High

3.Stimulus / Response Sequences

Stimulus (User or System Action)	System Response
User requests patient information	System verifies access permission
Permission is granted	System displays allowed information
Unauthorized access is detected	System denies access

4.Functional Requirements

- **FR-SAC-10:** The system shall protect confidential patient information.
- **FR-SAC-11:** The system shall restrict access to sensitive medical records.
- **FR-SAC-12:** The system shall ensure secure storage and transmission of patient data.

5. Non-Functional Requirements (NFR) Specification

5.1 Introduction to Non-Functional Requirements in DBMS

5.1.1 Functional vs. Non-Functional Requirements Scope

Functional requirements define the operational features CATMS must perform, whereas non-functional requirements (NFRs) establish the operational constraints, performance targets, and safety guarantees of those features.

	Functional Requirement (example)	Non-Functional Requirement (example)
Statement	"The system shall allow booking an appointment."	"The system shall reject overlapping bookings within 200 ms and maintain schedule integrity during concurrent submissions."
Scope	Feature presence and workflow execution.	Execution speed, transactional safety, and system resilience.

A system that books appointments but suffers from 30-second response latencies, race conditions under high load, or data corruption during server failure satisfies its functional specs while failing as a production-grade system.

5.1.2 Multi-Branch Context & NFR Rationale

CATMS operates across three concurrent physical branches (Colombo, Kandy, and Galle). Handling real-time financial transactions and clinical data across multiple locations introduces critical constraints that go beyond basic functional CRUD operations:

- **Financial Integrity:** Invoices and ledger entries must reconcile to the exact rupee. Minor discrepancies destroy managerial trust in system reports.
- **Data Privacy:** Even in early project phases without full RBAC, aggregate and statistical reports must strictly isolate personal health information (PHI).
- **System Availability:** Operational downtime directly prevents clinics from logging payments or retrieving patient history during business hours.
- **Concurrency Control:** Simultaneous reads and writes across three branches require strong transactional isolation to prevent dirty reads and race conditions.

- **Schema Extensibility:** Strategic expansion (such as adding a fourth branch or a new medical specialty) must occur via data insertion rather than schema refactoring.

These requirements justify using a relational database with declarative constraints, triggers, and ACID-compliant transactions over a lightweight file storage or spreadsheet model.

5.2 Comprehensive Non-Functional Requirements (Categorised)

5.2.1 Category: Performance & Query Latency

NFR-1.1: Management Report Response Time

All five management reports (branch summary, doctor revenue, outstanding balances, treatments per category, and insurance ratio) must return query results in under 2 seconds when tested against a realistic demo dataset of 1,000+ appointments.

Enforcement: Implemented via composite B-tree indexes aligned to each report's filter and group-by columns, such as (branch_id, scheduled_start) for daily branch summaries and (doctor_id, scheduled_start) for doctor revenue, along with a partial index on invoice(balance_due) WHERE balance_due > 0 so the outstanding balances query bypasses fully-paid invoices entirely.

NFR-1.2: Appointment Booking Response Time

The overlap check and insert operation for a new appointment must complete in under 300 milliseconds under typical load.

Enforcement: Executed via the trg_prevent_double_booking trigger, which runs an EXISTS query filtered by doctor_id and time range. The idx_appointment_doctor_time composite index enables PostgreSQL to perform an index scan directly on the target doctor's records rather than scanning the entire appointment table.

NFR-1.3: Sustained Performance as Data Grows

Query plans for all indexed queries must consistently utilize index scans rather than sequential scans, even as appointment and invoice volume grows into the tens of thousands of rows.

Enforcement: Validated during development and testing using PostgreSQL EXPLAIN ANALYZE to verify that the query planner actively selects defined indexes under scaled data conditions.

5.2.2 Category: Data Integrity & ACID Guarantees

NFR-2.1: Atomicity of Multi-Step Clinical Processes

Zero orphaned appointment_treatment rows or invoices with incorrect totals must exist following any failed or interrupted billing attempt, verified by passing an invalid treatment ID mid-call to confirm no partial data is persisted.

Enforcement: sp_CompleteAppointmentAndBill is implemented as a PostgreSQL PROCEDURE that executes its entire body as a single transaction. If any step raises an exception, PostgreSQL automatically rolls back all changes made during that call, preventing incomplete state updates.

NFR-2.2: Declarative Constraint Enforcement

The database must directly reject 100% of invalid data states regardless of the client application attempting the operation, including negative invoice balances, appointments ending before their start times, foreign keys referencing non-existent rows, and status values outside the defined set.

Enforcement: Enforced at the schema level via CHECK constraints (e.g., CHECK (scheduled_end > scheduled_start) and CHECK (status IN ('Scheduled', 'Completed', 'Cancelled'))), NOT NULL constraints on mandatory fields, and strict FOREIGN KEY constraints across all table relationships.

NFR-2.3: Double-Booking Prevention Under Concurrency

Concurrent overlapping booking attempts for the same doctor must yield a 0% success rate, verified through at least 100 simulated simultaneous insert attempts.

Enforcement: Executed via the trg_prevent_double_booking BEFORE INSERT OR UPDATE trigger running inside the same transaction as the insert. This leverages PostgreSQL row-level locking guarantees to ensure concurrent attempts are evaluated sequentially rather than against a stale view of the schedule.

NFR-2.4: Automatic Financial Consistency

invoice.balance_due must equal total_amount minus the sum of all payments for that invoice at all times, with zero discrepancies detectable by an audit query run at any moment.

Enforcement: Automated via the trg_update_invoice_on_payment AFTER INSERT trigger on the payment table, which recalculates total_paid, balance_due, and status immediately and automatically upon each payment entry.

5.2.3 Category: Data Security, Privacy & Data Minimisation

NFR-3.1: Report-Level Data Minimisation

The doctor revenue and insurance ratio reports must expose zero clinical or diagnosis fields, containing strictly the patient counts, monetary amounts, and dates required for their specific business purposes.

Enforcement: Implemented via dedicated SQL views (e.g., revenue summary views) that explicitly project only the required columns rather than querying base tables directly via SELECT *.

NFR-3.2: Forward-Compatible Least Privilege

A future authentication and role layer must be implementable without schema restructuring, allowing role additions (such as a report_viewer role restricted to SELECT operations on report views) via standard GRANT statements.

Enforcement: Architected using PostgreSQL role-based GRANT/REVOKE privileges scoped directly to views rather than underlying base tables, documented as a design decision for future extension while full authentication remains out of scope for this phase (SRS Section 2.8).

NFR-3.3: Sensitive Field Isolation

Clinical notes, consultation records, and emergency contact details must never appear in any statistical or aggregate report output, verified through a manual design review checklist.

Enforcement: Enforced through structural schema separation, keeping operational and financial columns (used for reporting) strictly distinct from personal and clinical columns (accessed exclusively via direct patient-record lookups).

5.2.4 Category: Reliability & Durability

NFR-4.1: Write Durability Through Crash Recovery

Zero committed transactions must be lost following a simulated abrupt database restart, such as forcibly terminating the database process mid-session and restarting it.

Enforcement: Managed through PostgreSQL Write-Ahead Logging (WAL), where every change is durably recorded to a sequential log before the transaction is marked as committed. Recovery after a crash replays this log to restore system state without relying on data being fully written to its final storage location beforehand.

NFR-4.2: Referential Consistency After Restart

All foreign key and constraint checks must pass 100% immediately following a forced restart triggered mid-transaction, leaving zero dangling references.

Enforcement: Managed via PostgreSQL WAL replay, which restores only fully committed transactions. Any transaction that did not reach COMMIT prior to the crash is automatically discarded in its entirety, enforcing atomicity guarantees (NFR-2.1) under system failure conditions.

NFR-4.3: Backup Recoverability

A full logical backup created via `pg_dump` must be restorable to an empty database instance, reproducing an identical schema structure and exact matching row counts.

Enforcement: Verified via a documented `pg_dump` and restore procedure included in the team's demo-preparation checklist, tested prior to final project evaluation.

5.2.5 Category: Usability & QA Testability

NFR-5.1: Full API Testability Without Custom UI

All functional requirements (100%) must be exercisable and independently verifiable through the FastAPI Swagger UI (/docs) alone, without writing any custom client code.

Enforcement: Supported at the backend level through FastAPI's automatic OpenAPI documentation generation, which directly reflects the route and schema definitions calling

underlying stored procedures and functions. This ensures the interactive test surface stays synchronized with database execution logic.

NFR-5.2: Self-Describing Error Messages

All business-rule violations raised by the database (100%), such as double-booking attempts, must surface a specific, human-readable message detailing which rule was broken.

Enforcement: Implemented using explicit RAISE EXCEPTION statements within database triggers and stored procedures (e.g., specifying the doctor and time range upon schedule overlap) rather than relying solely on generic constraint-violation codes.

NFR-5.3: Report Accessibility for Non-Technical Reviewers

All five reports must be viewable through the Streamlit/React dashboard by non-technical reviewers with **zero SQL knowledge**.

Enforcement: The dashboard directly executes and renders the identical indexed report queries specified in the companion Architecture document, ensuring visual report outputs strictly align with underlying database computations.

5.2.6 Category: Maintainability & Schema Scalability

NFR-6.1: Adding a Branch Without Redesign

Adding a new branch location (e.g., a fourth branch) must require exactly one INSERT statement into the branch table and zero ALTER TABLE statements anywhere in the schema.

Enforcement: Architected using a normalized branch lookup table referenced across the schema via branch_id foreign keys. Branch entities are never hardcoded into application logic, CHECK constraints, or table column definitions.

NFR-6.2: Adding a Speciality Without Redesign

Adding a new medical speciality must require a single INSERT statement into the speciality table and zero schema changes.

Enforcement: Facilitated through a normalized speciality lookup table combined with a doctor_specialty many-to-many bridge table (Architecture Document, Section 3), allowing flexible entity expansion through data insertion rather than structural modifications.

NFR-6.3: Third Normal Form (3NF) Compliance

All core tables must satisfy Third Normal Form (3NF) as confirmed by a documented walkthrough, with any intentional exception (such as the price_snapshot field on appointment_treatment) explicitly justified rather than accidental.

Enforcement: Achieved through deliberate schema design reviewed against 3NF rules during Milestone 1 (Architecture Document, Sections 3 and 7). The price_snapshot column serves as the single intentional, documented departure from strict normalization, preserved because historical invoice accuracy outweighs minor data redundancy.

NFR-6.4: Centralised, Editable Business Logic

Changing a core business rule (e.g., requiring a 10-minute buffer between appointments instead of a zero gap) must require editing exactly one trigger function, with zero application-layer code changes.

Enforcement: Achieved by encapsulating business rules directly within the database itself (using triggers, stored procedures, and database functions) rather than duplicating logic inside the FastAPI layer, maintaining a single, authoritative source of truth for every rule.

5.3 Summary NFR Metrics Table

NFR ID	Category	Metric / Target	Enforcement Mechanism	Verification Method
NFR-1.1	Performance	Reports return in < 2s on 1,000+ appointment dataset	Composite + partial B-tree indexes	Time each report query; run EXPLAIN ANALYZE
NFR-1.2	Performance	Booking overlap-check < 300ms	idx_appointment_doctor_time index	Time booking API calls under load
NFR-1.3	Performance	Index scans (not sequential) as data grows	Query plan review during development	EXPLAIN ANALYZE on all report queries
NFR-2.1	Data Integrity	Zero orphan rows after a failed billing attempt	sp_CompleteAppointmentAndBill as an atomic PROCEDURE	Force a mid-call error; confirm full rollback
NFR-2.2	Data Integrity	100% rejection of invalid states at schema level	CHECK, NOT NULL, FOREIGN KEY constraints	Attempt each invalid insert directly via SQL
NFR-2.3	Data Integrity	0% successful overlapping bookings under concurrency	trg_prevent_double_booking trigger	Simulate 100+ concurrent conflicting inserts
NFR-2.4	Data Integrity	balance_due always exactly correct	trg_update_invoice_on_payment trigger	Run an audit query comparing balance vs. payment sum
NFR-3.1	Security & Privacy	Reports expose zero clinical/diagnosis fields	Purpose-built report views	Inspect view definitions and sample output
NFR-3.2	Security & Privacy	Auth layer addable without schema redesign	Role-scoped GRANT/REVOKE design	Design review checklist
NFR-3.3	Security & Privacy	Sensitive fields never appear in aggregate reports	Structural separation of clinical vs. operational columns	Manual review of every report query

NFR ID	Category	Metric / Target	Enforcement Mechanism	Verification Method
NFR-4.1	Reliability	Zero committed transactions lost after crash	PostgreSQL WAL	Kill DB process mid-session; restart; verify data
NFR-4.2	Reliability	100% referential integrity after restart	WAL replay of committed transactions only	Force restart mid-transaction; run FK checks
NFR-4.3	Reliability	Backup restores to identical schema/row counts	pg_dump / restore	Restore backup to empty DB; compare counts
NFR-5.1	Usability	100% of FRs testable via Swagger UI	FastAPI auto-generated /docs	Walk every FR checklist item through /docs
NFR-5.2	Usability	100% of rule violations return a clear message	Explicit RAISE EXCEPTION text	Trigger each business rule; read the error
NFR-5.3	Usability	Reports viewable with zero SQL knowledge	Streamlit/React dashboard	Non-technical team member reviews dashboard
NFR-6.1	Maintainability	New branch = 1 INSERT, 0 ALTER TABLE	branch lookup table	Insert a 4th branch; confirm no schema change needed
NFR-6.2	Maintainability	New specialty = 1 INSERT, 0 schema change	specialty + doctor_specialty tables	Insert a new specialty; confirm existing queries unaffected
NFR-6.3	Maintainability	All core tables satisfy 3NF (exceptions documented)	Reviewed schema design	Manual 3NF walkthrough vs. Architecture doc
NFR-6.4	Maintainability	Rule changes = 1 trigger edit, 0 app code changes	Centralized DB-layer business logic	Change one rule; confirm no FastAPI code touched

6. Other Requirements and Appendixes

This section closes out the SRS with material that supports Sections 1–5 rather than stating new obligations of its own: a traceability matrix confirming every clause of the MedSync client brief is covered by at least one functional requirement, sample layouts for the five management reports, a representative set of QA test cases for Appendix A referenced in Sections 1.1 and 3.1.1, a short set of additional requirements not naturally placed in Sections 1–5, and a supplementary glossary of database-specific terms used in Section 5.

6.1 Traceability Matrix

The table below maps each in-scope clause of the client brief (Section 1.2) to the functional requirements that satisfy it and, where relevant, the non-functional requirements that govern how well it must be satisfied. Every clause has at least one FR; no FR exists without a traceable clause.

#	Client Brief Clause (Section 1.2, In Scope)	Satisfied By (FRs)	Related NFR(s)
1	Registration and maintenance of branches, staff, and assignment of one branch manager per branch	FR-BM-01–10, FR-SDM-01–03	NFR-6.1
2	Doctor register: exactly one branch per doctor, one or more specialties	FR-SDM-04–14	NFR-6.2
3	Patient registration at any branch; single logical record retrievable from every branch; personal, emergency contact and insurance details	FR-PRM-01–17	NFR-3.1, NFR-3.3
4	Booking, rescheduling and cancellation of appointments with hard prevention of overlap for the same doctor	FR-AM-01–07, FR-AM-12–13	NFR-1.2, NFR-2.3
5	Creation of emergency walk-in appointments without a prior booking	FR-AM-08–09	—
6	Recording of consultation notes and one or more catalogued treatments against a completed appointment	FR-TCM-01–06, FR-TCM-10–12	NFR-2.1
7	Maintenance of a treatment catalogue with service code, category and price	FR-TCM-07–09	NFR-6.4
8	Automatic generation of an invoice from a completed appointment's treatments	FR-BPM-01–09	NFR-2.1, NFR-2.2
9	Acceptance of full or partial payments and continuous tracking of the outstanding due	FR-BPM-10–19	NFR-2.4

10	Lodging and settlement of insurance claims with partial/full reimbursement per policy terms	FR-ICM-01-12	NFR-2.2
11	Production of the five management reports	FR-RA-01-10	NFR-1.1, NFR-3.1, NFR-5.3
12	Database-level enforcement of business rules (keys, constraints, procedures, functions, triggers, ACID)	FR-DB-01-31	NFR-2.1-2.4, NFR-4.1-4.3, NFR-6.3
13	Thin interface sufficient for QA to exercise every feature (per constraint C-3)	FR-EI-01-06	NFR-5.1, NFR-5.2
14	Access restricted according to the six user classes of Section 2.3	FR-SAC-01-12, FR-EI-03	NFR-3.2
15	Insurance claims exported as a file for manual submission (no real-time interchange in Phase 1)	FR-EI-13	—

6.2 Sample Report Layouts

The layouts below illustrate the column structure of each of the five management reports specified in Section 4.8. Each report is filterable by branch, doctor and/or date range as noted in Section 3.1.2, and is rendered from a dedicated report view rather than a base-table query, per NFR-3.1 and FR-EI-11.

6.2.1 Branch-wise Appointment Summary (FR-RA-01, FR-RA-02)

Branch	Date	Scheduled	Completed	Cancelled	Total
Colombo	2026-07-27	18	14	3	35
Kandy	2026-07-27	12	10	1	23

6.2.2 Doctor Revenue Report (FR-RA-03, FR-RA-04)

Doctor	Branch	Consultations	Treatments	Revenue (LKR)
Dr. Perera	Colombo	42	17	142,500.00
Dr. Fernando	Galle	35	9	104,300.00

6.2.3 Outstanding Balance Report (FR-RA-05, FR-RA-06)

Patient	Invoice ID	Total (LKR)	Paid (LKR)	Outstanding (LKR)
W.A. Silva	INV-00231	8,500.00	3,000.00	5,500.00

6.2.4 Treatment Category Report (FR-RA-07, FR-RA-08)

Category	Date Range	Treatment Count
Diagnostic (X-Ray, ECG)	2026-07-01 – 2026-07-27	61
Minor Procedure (Dressing, Injection)	2026-07-01 – 2026-07-27	48

6.2.5 Insurance Coverage Report (FR-RA-09, FR-RA-10)

Patient	Invoice ID	Insurance Covered (LKR)	Out-of-Pocket (LKR)
K.D. Jayasuriya	INV-00318	5,600.00	1,400.00

6.3 Appendix A - Sample QA Test Cases

Appendix A, referenced in Sections 1.1 and 3.1.1, is the starting set of test cases the QA team executes through the Swagger UI (/docs) to verify that the database-enforced business rules of Section 4.9 and the measurable NFR targets of the companion NFR specification actually hold. The table gives a representative subset; the QA team is expected to extend it to full coverage of Section 4.

ID	Verifies	Steps	Expected Result
TC-01	FR-AM-12/13, NFR-2.3	Book an appointment for Dr. X 10:00–10:30; attempt a second booking for Dr. X 10:15–10:45	Second booking rejected with a specific overlap error (NFR-5.2); first booking unaffected
TC-02	NFR-2.2	Attempt to insert an appointment with scheduled_end earlier than scheduled_start	Insert rejected by the CHECK constraint
TC-03	FR-PRM-05	Register a patient with a given national identity number, then attempt to register a second patient with the same number	Second registration rejected as a duplicate
TC-04	FR-BPM-05	Attempt to generate an invoice for an appointment still in 'Scheduled' or 'Cancelled' status	Invoice generation refused; no invoice row created
TC-05	FR-BPM-14–16, NFR-2.4	Record two partial payments against one invoice	balance_due after each payment equals total_amount minus the sum of payments recorded so far

TC-06	NFR-2.2, NFR-2.4	Attempt to record a payment larger than the current outstanding balance	Payment rejected; balance_due never goes negative
TC-07	FR-ICM-04–06	Lodge a claim for a policy with an 80% coverage percentage against an eligible treatment, then against a non-claimable consultation line	Reimbursement calculated only for the eligible line, capped by the policy's annual ceiling; consultation line excluded per its is_claimable flag
TC-08	FR-BM-07	Assign a manager to a branch that already has an active manager	Assignment rejected; existing manager assignment unchanged
TC-09	FR-DB-02, NFR-2.2	Attempt to create an appointment referencing a non-existent doctor_id	Insert rejected by the foreign-key constraint
TC-10	NFR-1.1	Request each of the five management reports against a seeded dataset of 1,000+ appointments	Every report returns in under 2 seconds
TC-11	NFR-2.3	Fire 100 simultaneous booking attempts for the same doctor and overlapping time slot	Exactly one booking succeeds; all others are rejected with no double-booked row present afterward
TC-12	NFR-5.2	Trigger each documented business-rule violation in turn (overlap, duplicate patient, overpayment, invalid status)	Each returns a distinct, human-readable message naming the rule broken, not a generic error code

6.4 Additional Requirements Not Elsewhere Categorized

The following requirements are necessary for Phase 1 acceptance but did not fit naturally under Sections 3–5.

- OR-01: Patient health data shall be encrypted at rest and access-logged, and retained for a documented period, consistent with the Personal Data Protection Act No. 9 of 2022 (Ref. [7]) and constraint C-5.
- OR-02: A seed script populating representative dummy data (branches, staff, patients, appointments, treatments, invoices, insurance policies) shall be delivered alongside the schema, per constraint C-4, and may be applied by direct SQL rather than through the UI.
- OR-03: Monetary columns shall be declared DECIMAL(12,2) throughout; no monetary value shall be stored or computed using a floating-point type, per constraint C-6.

- OR-04: All timestamp columns shall store Asia/Colombo local time with no daylight-saving adjustment, per constraint C-7; no timezone-conversion logic is required.
- OR-05: The delivered schema shall run unmodified, other than engine-specific procedure/trigger syntax, on both PostgreSQL 15 and MySQL 8.0, per constraint C-9 and FR-EI-10.

6.5 Supplementary Glossary

The core definitions and acronyms of Section 1.3 are not repeated here. The terms below are used only in the NFR specification (Section 5) and are defined for readers who consult Section 6 without having first read Section 5 in full.

Term	Definition
WAL (Write-Ahead Logging)	A PostgreSQL durability mechanism that records every change to a sequential log before a transaction is reported as committed, allowing crash recovery by replaying the log.
View	A stored, named SQL query that report and dashboard code selects from directly, so it can expose only the columns a report needs (NFR-3.1) without touching base tables.
GRANT / REVOKE	SQL statements that assign or withdraw a database role's privileges on a specific table or view, the mechanism assumed by NFR-3.2 for adding a future least-privilege access layer.
Sequential scan	A query execution plan that reads every row of a table; the opposite of an index scan and the condition NFR-1.3 requires the query planner to avoid as data volume grows.
Price snapshot	The unit price copied onto an appointment_treatment line at the time of billing, kept intentionally redundant with the treatment catalogue so historical invoices remain accurate if catalogue prices later change (NFR-6.3).